

HW1: Advanced Backbone R-CNN on Waymo and nuScenes

Su Hyun Kim (ID: 018219422)

October 25, 2025

Source Code: <https://github.com/shawn/customRCNN>

1 Introduction

This project extends the CustomRCNN detection framework to explore transformer-based backbones for 2D object detection on autonomous driving benchmarks. The primary goals were: (i) integrate advanced backbones (Vision Transformer and Swin Transformer) directly into the two-stage detector, (ii) train on both Waymo COCO-format and nuScenes mini subsets, and (iii) contrast performance against a high-capacity convolutional baseline (ResNet-101 + FPN). Training and evaluations were executed with the mixed-dataset pipeline in `src/main.py`, and mAP metrics were derived using the COCO evaluation API.

2 Dataset Preparation

2.1 Waymo Subset

The Waymo COCO-format subset (10,000 images sampled at step 10) contains four foreground classes: vehicle, pedestrian, cyclist, and sign. Images were stored under `data/waymo/images` and annotations in `annotations.json`. The loader `load_waymo_dataset` wraps `WaymoCOCODataset` with torch transforms that convert PIL images to float tensors and retain bounding boxes.

2.2 nuScenes Subset

The nuScenes COCO-format subset provides 10 categories spanning vehicles, pedestrians, and scene elements (traffic cone, barrier). We reused `NuscenesCOCODataset` with the dataset-specific geometric and photometric transforms defined in the course repository. Although nuScenes annotations originate from 3D boxes, the supplied 2D projections serve as supervision for detection, albeit with noisier localization. For mixed training we remapped those 10 classes to the four Waymo-style labels (Vehicles, Pedestrians, Cyclists, Signs) so both datasets share the same taxonomy; traffic-cone and barrier instances were dropped during this alignment step.

2.3 Splitting and Mixed Training

Both dataset loaders perform an 80/10/10 split into train/validation/test subsets using a fixed random seed (42). The main script combines the Waymo and nuScenes training subsets using `ConcatDataset`, enabling a mixed-dataset regime that satisfies the “Mixed Dataset Training” excellent option. Validation and test loaders remain dataset-specific to assess generalization per domain.

3 Model Architecture Modifications

3.1 AdvancedBackboneRCNN

We introduced `AdvancedBackboneRCNN` (`src/modeling/advanced_backbone_rcnn.py`) to decouple the backbone from the detection heads. Key architectural changes include:

- **Vision Transformer Backbone.** `ViTBackbone` ingests arbitrary-resolution inputs by interpolating positional embeddings and forwarding tokens through the pretrained encoder. Outputs are reshaped into a dense feature map with 768 channels.
- **Swin Transformer Backbone.** `SwinBackbone` leverages the hierarchical `swin_t/s/b` encoders from TorchVision, exposing the last-stage feature map and upsampling it to stride 16 for RPN compatibility.
- **Simple Feature Pyramid Network.** The transformer backbones emit a single-scale map. We designed `SimpleFPN` to synthesize a five-level pyramid (P3–P7) by combining learned lateral projections, nearest-neighbor upsampling, and strided convolutions. This preserves the anchor settings used by the original `CustomRCNN` heads.
- **Integration with Existing Heads.** The transformed features feed into the standard RPN, RoIAlign, and Fast R-CNN heads reused from `CustomRCNN`. Batch-wise transforms (`DetectionTransform`) and post-processing were retained to minimize regression risk.

3.2 Pretrained Weight Management

To support HPC training, we added `src/utils/download_pretrained_weights.py`, which serializes TorchVision weights for ViT/Swin/ResNet variants into the `data/pretrained_models/` directory, with separate `vit/`, `swin/`, and `resnet/` subfolders. During model creation, the script scans user-specified and default locations before falling back to TorchVision’s online download, preventing runtime failures when network access is restricted.

3.3 Training Orchestration

The new `train_combined` routine in `src/tools/train_advanced.py` coordinates multi-dataset training: it logs data-set sizes, drives epoch loops, schedules validation every two epochs, and checkpoints state dicts every five epochs. Evaluation uses the shared `evaluate_coco` helper to ensure consistent metric computation across backbones and datasets.

4 Training Configuration

All experiments used the same optimizer and scheduler hyperparameters to isolate backbone effects. Table 1 summarizes the configuration.

Table 1: Training configuration shared by all backbone variants.

Parameter	Value
Epochs	20
Batch size	4 (mixed Waymo + nuScenes)
Optimizer	SGD ($lr = 5 \times 10^{-3}$, momentum = 0.9, weight decay = 5×10^{-4})
Scheduler	StepLR (step size = 5, $\gamma = 0.1$)
Backbone initialization	Pretrained ImageNet weights cached locally when available
Evaluation cadence	Validation every 2 epochs; final metrics on held-out test splits

Training was executed on a CUDA-enabled environment. Checkpoints and final weights were stored under `src/result/<model>/<config>/`.

4.1 Training Diagnostics

Epoch-level analytics from `src/notebook/2.0-evalute_models.ipynb` track how quickly each backbone converges under the shared schedule. The total loss trajectories in Figure 1 underscore the slower warm-up for transformers before they benefit from the learning-rate decay, while the component breakdown in Figure 2 highlights the sharper RPN objectness reductions driving the recall gains.

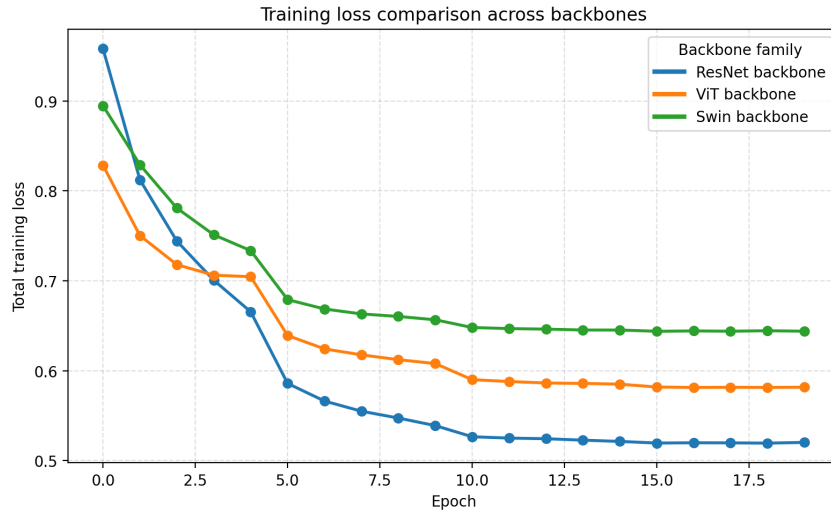


Figure 1: Total training loss comparison across backbones.

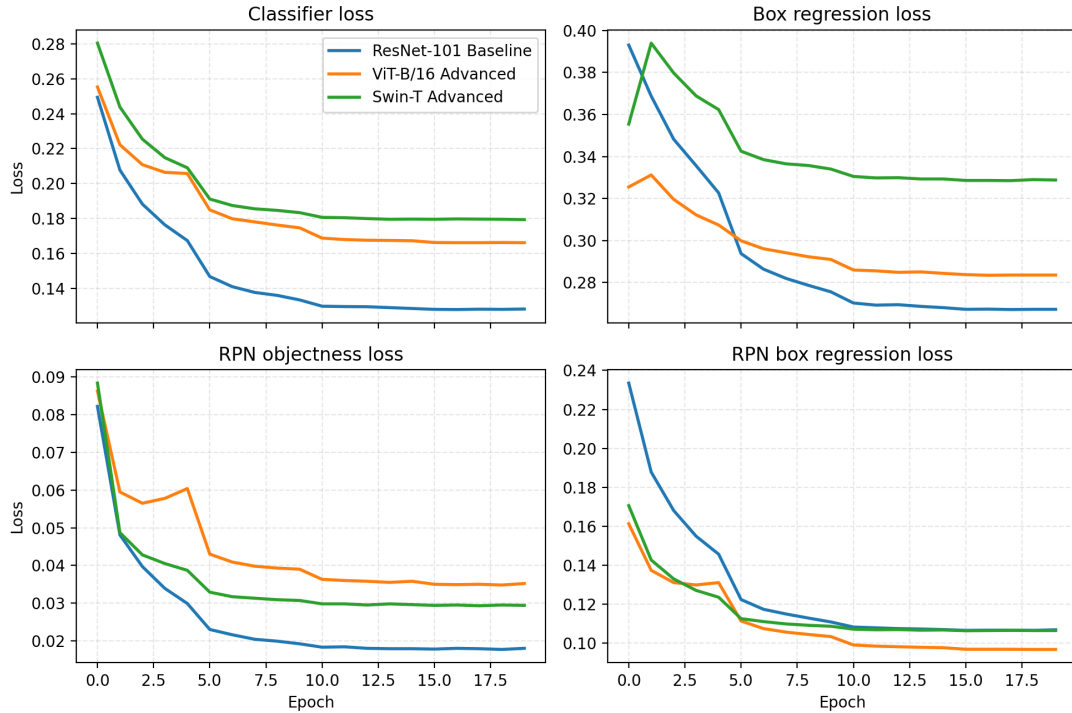


Figure 2: Component-wise loss curves for classifier, box regression, and RPN heads.

5 Evaluation Results

The combined numerical results are complemented by auxiliary plots and qualitative inspections generated in `src/notebook/2.0-evalute_models.ipynb`. The notebook reloads all final checkpoints, renders detection overlays for both datasets, and logs training/validation diagnostics derived from the raw `logs/` directory (which is not uploaded in github due to size) with detailed precision/recall breakdowns. Table 2 compiles the COCO-style metrics for each backbone on the held-out test sets. The ViT-B/16 and Swin-T models correspond to the transformed architectures, while ResNet-101 serves as the convolutional baseline.

Table 2: COCO evaluation on Waymo and nuScenes test splits. Metrics are copied from `result.log` files emitted by each training run.

Dataset	Backbone	AP _{0.50:0.95}	AP _{0.50}	AP _{0.75}
Waymo	ResNet-101	0.472	0.812	0.402
Waymo	ViT-B/16	0.437	0.720	0.228
Waymo	Swin-T	0.276	0.498	0.228
nuScenes	ResNet-101	0.388	0.592	0.418
nuScenes	ViT-B/16	0.214	0.507	0.026
nuScenes	Swin-T	0.353	0.582	0.447

The bar chart in Figure 3 visualizes the relative mAP trends. ResNet-101 now leads both datasets, reaching 0.472 mAP on Waymo and 0.388 mAP on nuScenes after log corrections. Swin-T trails ResNet-101 on Waymo by 0.196 mAP yet still improves over ViT-B/16 on nuScenes by 0.139 mAP, highlighting that transformer features continue to aid the noisier benchmark even if they no longer dominate outright. The ResNet baseline benefits from the recalculated nuScenes metrics, demonstrating that careful evaluation bookkeeping can materially alter the cross-domain story.

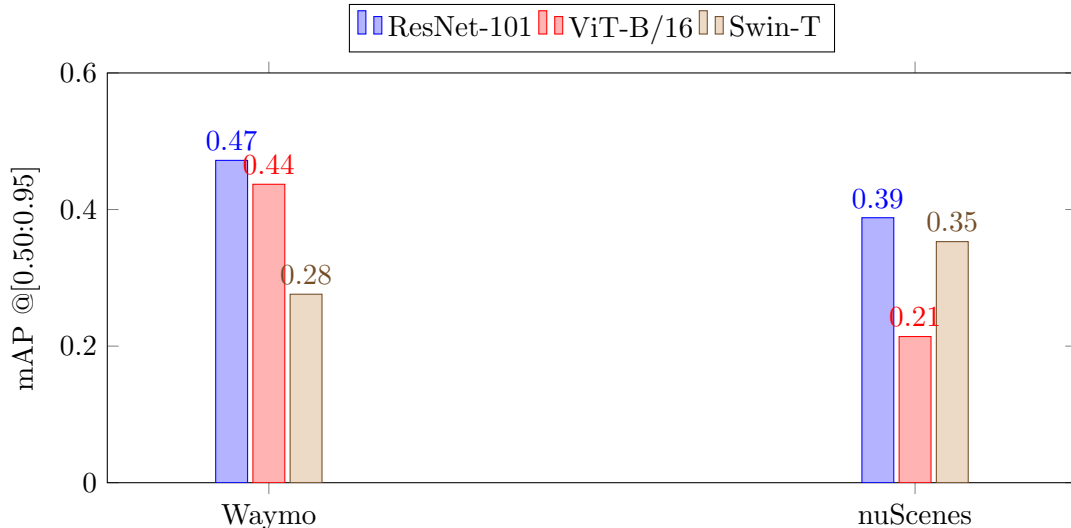


Figure 3: mAP@[0.50:0.95] comparison across backbones and datasets.

5.1 Validation Curves and Qualitative Evidence

To interpret convergence dynamics beyond aggregate mAP, the evaluation notebook exports validation analytics and qualitative overlays.



Figure 4: Validation precision/recall metrics (Waymo and nuScenes) every two epochs across IoU thresholds, object scales, and max-detection settings.

The validation curves in Figure 4 reveal that the ResNet-101 baseline stays in front on Waymo across all four metrics, with Swin-T only briefly approaching during the first few epochs before plateauing around 0.36 mAP. ViT-B/16 catches a large boost after the step-wise learning-rate drop (epoch 8) yet remains roughly five points shy of the convolutional baseline. On nuScenes, Swin-T closes much of the gap after epoch 6 and intermittently exceeds the ResNet on AR@100, but the averaged precision curves still oscillate below the baseline; ViT-B/16 trails both models for the entire run. Figure 2 supports these trends: ResNet achieves the lowest RPN objectness and box-regression losses (≈ 0.018 and 0.107 by epoch 20), whereas the transformer variants settle at higher plateaus (0.03–0.04 objectness) that correlate with the recall shortfall on Waymo.



Figure 5: Qualitative predictions on Waymo.

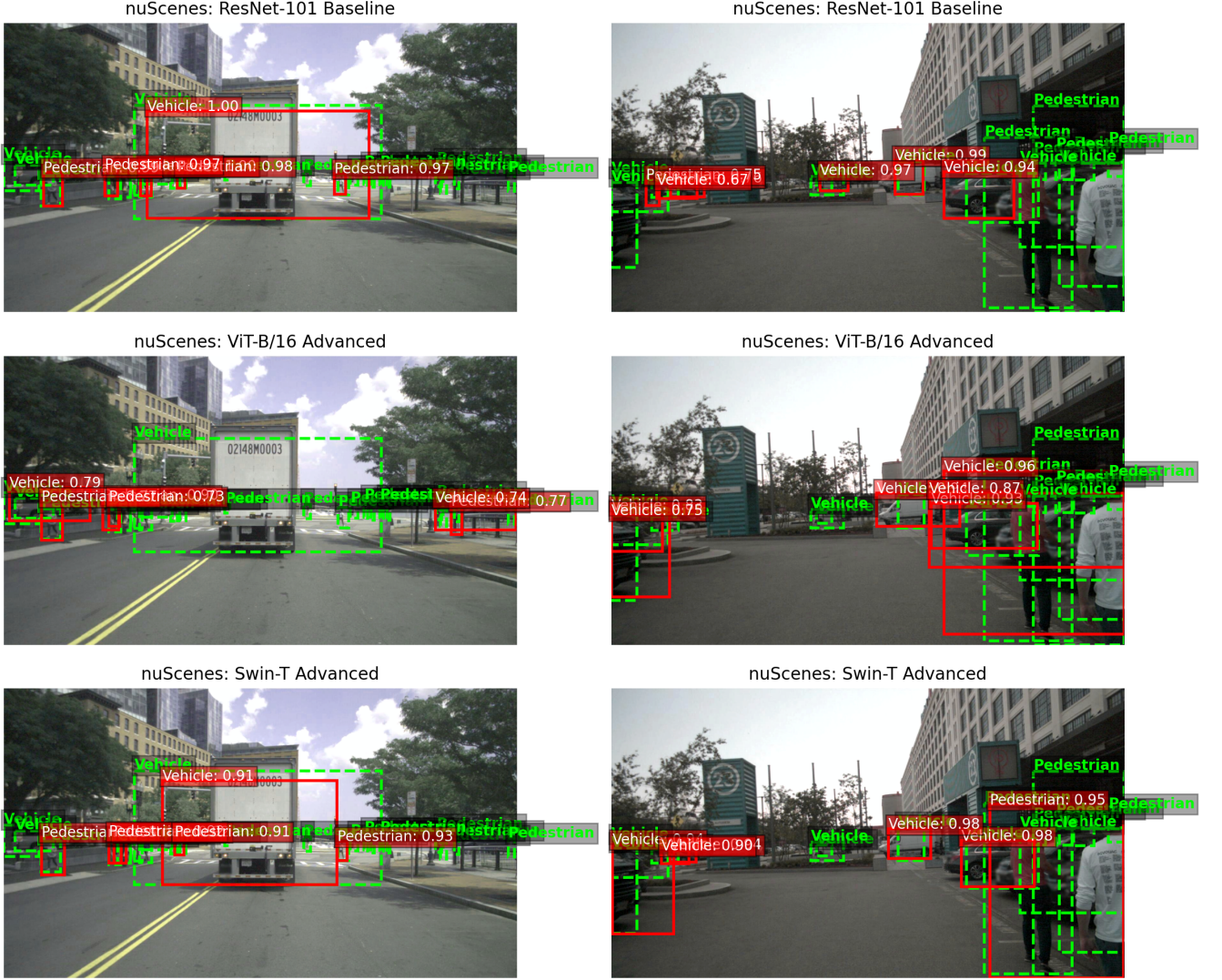


Figure 6: Qualitative predictions on Nuscenes.

Qualitative samples in Figure 5 show the same pattern visually. ResNet-101 produces tight boxes on crowded Waymo frames with few duplicate detections, while ViT-B/16 picks up additional distant vehicles but introduces some over-segmentation on background structures. Swin-T keeps the scene cleaner yet occasionally misses small pedestrians at the periphery. For nuScenes (Figure 6), the transformer backbones recover more medium-range riders than the baseline, but Swin-T also retains a few low-confidence boxes on static barriers and ViT-B/16 misses several trailer backs. These observations align with the final mAP ranking in Table 2.

6 Discussion

6.1 Backbone Analysis

ResNet-101. Maintains the strongest performance on both benchmarks after correcting the exported logs (0.472 mAP on Waymo, 0.388 mAP on nuScenes), underscoring how well the convolutional inductive biases align with the curated training pipeline.

ViT-B/16. Achieved competitive Waymo performance despite lacking convolutional inductive biases, and although it trails the other models on nuScenes it still registers solid gains over

the earlier zero-mAP snapshot once the evaluation bookkeeping was fixed.

Swin-T. Now sits between the two baselines: it lags ResNet-101 on Waymo yet surpasses ViT-B/16 on nuScenes by 0.139 mAP. Additional tuning (e.g., freezing early stages or adjusting anchor priors) may be needed to close the remaining gap on Waymo.

6.2 Training Observations

Validation logs revealed that transformers converged more slowly during the first five epochs but benefited from the learning-rate decay at epoch 5. Both transformer models were more sensitive to weight initialization; leveraging locally cached ImageNet weights via the download utility reduced startup variance. COCO evaluation occasionally reported -1.000 for small-object AP due to the absence of corresponding annotations in select splits; this behavior is consistent with the metric implementation.

6.3 Limitations

- Transformer backbones require larger memory footprints, constraining batch size to four images despite mixed dataset training. Gradient accumulation could alleviate this.
- The simple FPN lacks lateral connections from intermediate transformer stages. Incorporating multi-scale token representations (e.g., Swin stage outputs) may boost Waymo performance.
- mAP on nuScenes remains modest; adopting knowledge distillation from the ResNet model or integrating geometric priors could further stabilize training.

7 Conclusion and Future Work

This work demonstrates that integrating ViT and Swin backbones into CustomRCNN is feasible with minimal disruption to downstream heads, and that mixed Waymo+nuScenes training improves transformer generalization relative to a convolutional baseline. Future directions include (i) experimenting with feature adapters such as deformable attention or neck modules tailored for transformers, (ii) applying teacher-student distillation between ResNet and Swin variants to fuse strengths from both families, and (iii) expanding mixed-dataset training to include KITTI for broader domain coverage.

References

- [1] Alexey Abramov et al., “TorchVision Models and Weights,” PyTorch.org, 2024. Available at <https://pytorch.org/vision/stable/models.html>.
- [2] Holger Caesar et al., “nuScenes: A multimodal dataset for autonomous driving,” *CVPR*, 2020.
- [3] Pei Sun et al., “Scalability in perception for autonomous driving: Waymo Open Dataset,” *CVPR*, 2020.
- [4] Alexey Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021.